ENCE260: Embedded Systems

Lecture 4: Timing

How do we control how fast things happen?

Revision

How would you configure and use (in code) a switch connected with a pull-up resistor to `PB5`?

First, configure as an input because it's a switch:

```
DDRB &= ~(1<<5);
```

To use, we need to check its value.

It's connected to a pull-up, so the switch is active-low:

```
if (PINB & (1<<5) == 0) {
    // Button is pressed.
```

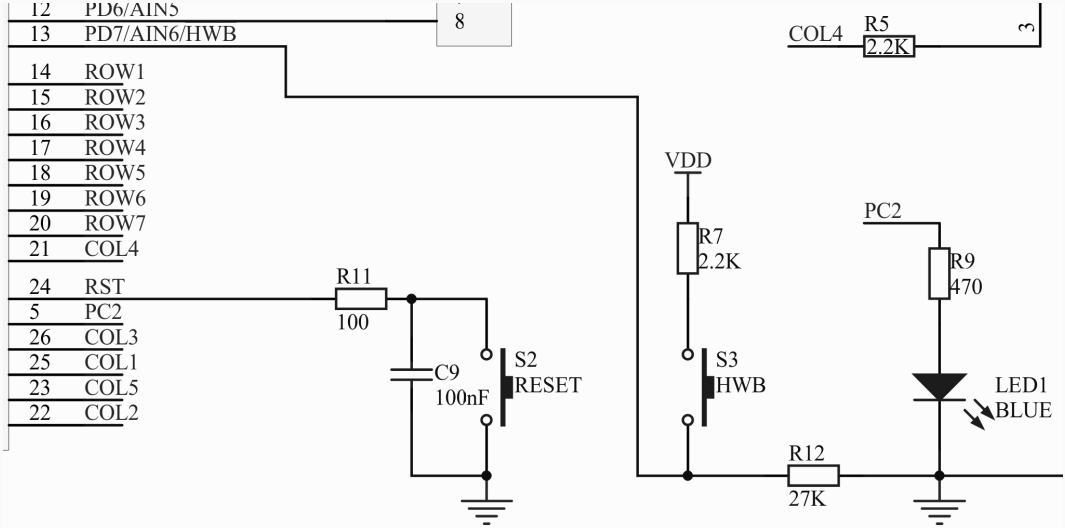
Revision

The Fun Kit has a switch connected to `PD7`. Which of the following configurations does this switch have?

1. Internal pull-up resistor.
2. External pull-up resistor.
3. External pull-down resistor. ✓
4. No configuration resistor.

The switch is therefore active high.

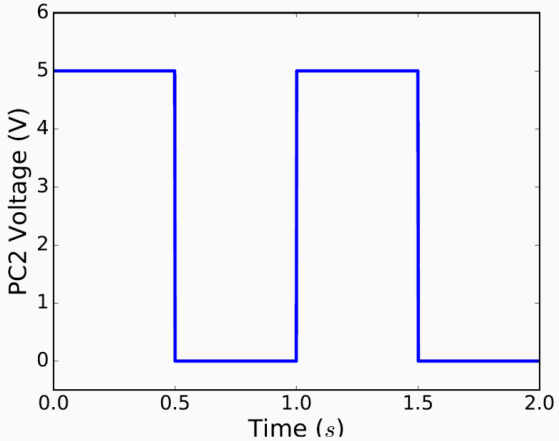
Revision

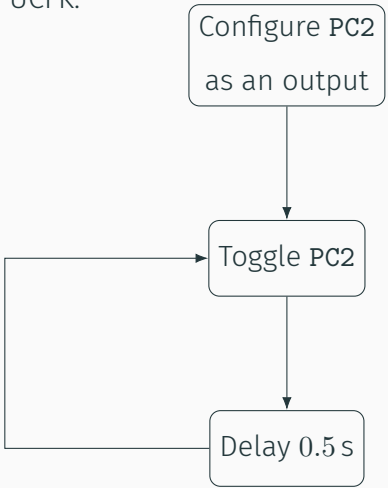

12 PD6/AIN5
13 PD7/AIN6/HWB
14 ROW1
15 ROW2
16 ROW3
17 ROW4
18 ROW5
19 ROW6
20 ROW7
21 COL4
24 RST
5 PC2
26 COL3
25 COL1
23 COL5
22 COL2
8
R11
100
C9
100nF
S2
RESET
COL4
R5
2.2K
3
VDD
R7
2.2K
S3
HWB
PC2
R9
470
LED1
BLUE
R12
27K

Flashing an LED

- Say we want to flash the user LED at 1 Hz on the UCFK.

$$f\,[\text{Hz}] = \frac{\text{cycles}}{\text{second}},\ T\,[\text{s}] = \frac{1}{f}.$$





“Blinky” Implementation

```c
#include "system.h"
#include "delay.h"
#include <avr/io.h>
#include <stdbool.h>

int main(void)
{
    system_init();

    // Set up PC2 as an output.
    DDRC |= (1 << 2);

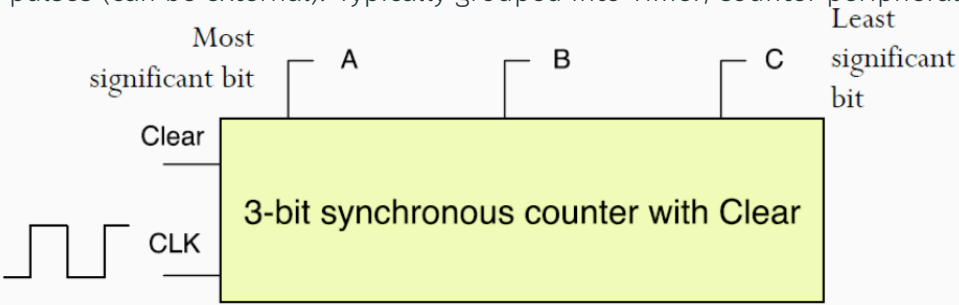
    while (true) {
        // Toggle LED state.
        PORTC ^= (1 << 2);

        // Wait for 500-ish
    milliseconds.
        delay_ms(500);
    }
}
```

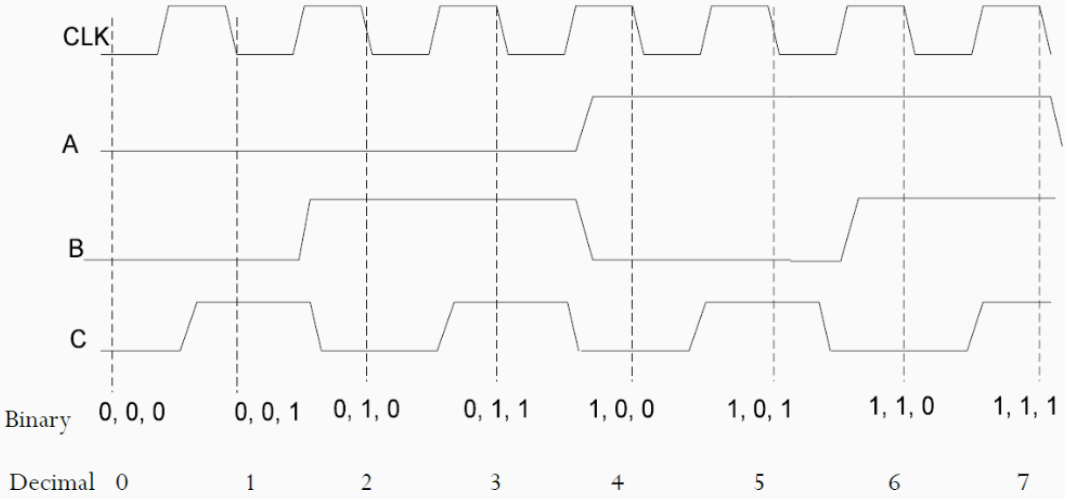
- Here, `delay_ms()` is some function that delays for a specified number of milliseconds.
- Some considerations:
  - It occupies the CPU, so we can’t do anything else.
  - `delay_ms()` has to be microcontroller-specific.
  - Will take > 500 ms due to overhead.

Counters

- A counter stores the number of times a particular event has occurred.
- Counter circuits are usually made from flip-flops connected in cascade.
- Synchronous counters are synchronised with a clock signal.
- A timer uses the frequency of the internal clock, whereas a counter counts pulses (can be external). Typically grouped into Timer/Counter peripherals.



Counter Waveform

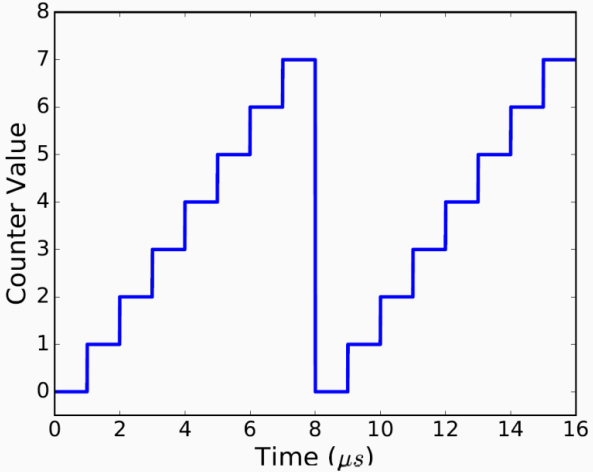


Timer/Counters

- Most counters can be optionally configured to take input from the microcontroller clock.
- This means they can act as a counter when connected to an external signal, or a timer if connected to the internal clock.
- This is a type of peripheral (outside the central CPU) called a timer/counter.
- Microcontrollers will often have multiple timer/counters, so different ones can do different jobs.

Timer/Counter Example

Consider an $N = 3$ bit counter incremented by a 1 MHz clock.



- $f_{\text{clk}} = 1 \times 10^6$ Hz.
- Maximum value $= 2^N - 1 = 7$.
- Resolution $\Delta t = \frac{1}{f_{\text{clk}}} = 1$ µs.
- Max delay = Period = $T = 2^N \cdot \Delta t$

$$T = 2^3 \cdot 1\ \mu\text{s},$$

$$T = 8\ \mu\text{s}.$$

Timer/Counter Overflow

- At the system clock frequency (8 MHz on the UCFK4), the counter will overflow back to 0 very rapidly.
- For example, for the 16-bit timer/counter `TCNT1`:

  Resolution: $\Delta t = \frac{1}{f_{\text{clk}}} = 1.25 \times 10^{-7}$ s.

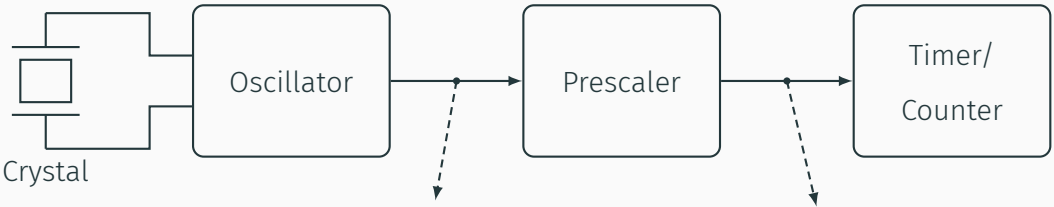
  $N = 16$ bits, $2^{16} = 65536$ levels.

  Overflow when $T = 65536 \times 1.25 \times 10^{-7} = 8.192$ ms.

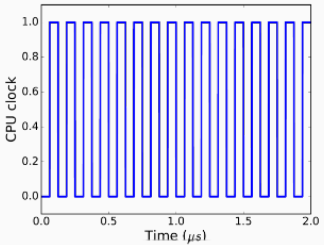
- We can slow down the clock speed by dividing the clock speed by an integer using a *prescaler*. This is done in hardware with another counter.

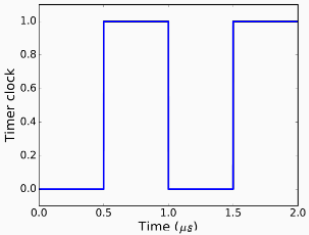
Prescaling a Timer

For our $8\,\text{MHz}$ CPU clock and a prescaler of 8:



$f_{\text{clk}} = 8\,\text{MHz}$

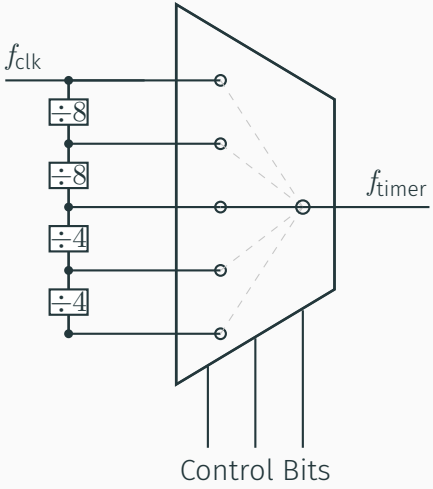




$f_{\text{timer}} = 1\,\text{MHz}$

Prescaler on the AVR

- Microcontrollers normally have a limited choice of values for the prescaler.
- On the ATmega32U2, the possible prescaler values are 1, 8, 64, 256, and 1024.



AVR Prescaler Configuration

16.11.2 TCCR1B – Timer/Counter1 Control Register B

Bit	7	6	5	4	3	2	1	0	
(0x81)	ICNC1	ICES1	–	WGM13	WGM12	CS12	CS11	CS10	TCCR1B
Read/Write	R/W	R/W	R	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

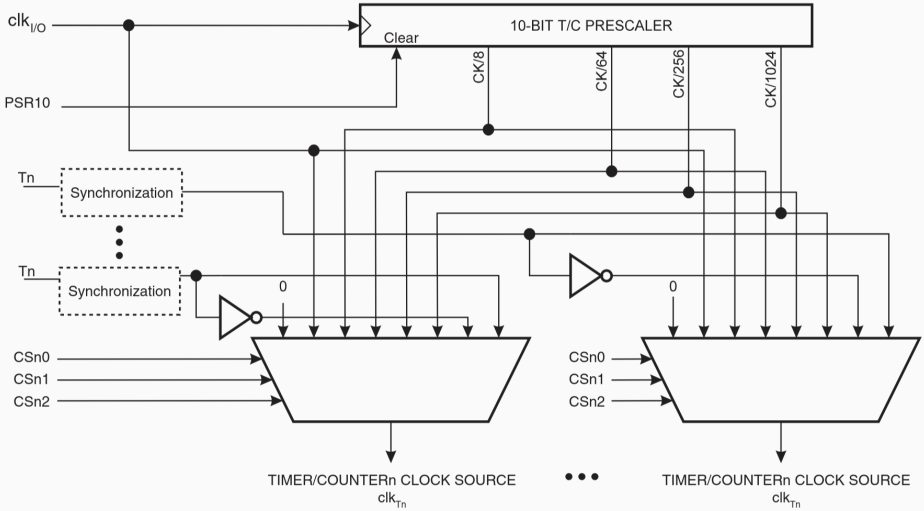
- **Bit 2:0 – CSn[2:0]: Clock Select**

The three clock select bits select the clock source to be used by the Timer/Counter, see Figure 15-1 and Figure 15-2.

Table 16-5. Clock Select Bit Description

CSn2	CSn1	CSn0	Description
0	0	0	No clock source. (Timer/Counter stopped)
0	0	1	$clk_{I/O}$/1 (No prescaling
0	1	0	$clk_{I/O}$/8 (From prescaler)
0	1	1	$clk_{I/O}$/64 (From prescaler)
1	0	0	$clk_{I/O}$/256 (From prescaler)
1	0	1	$clk_{I/O}$/1024 (From prescaler)
1	1	0	External clock source on Tn pin. Clock on falling edge
1	1	1	External clock source on Tn pin. Clock on rising edge

Figure 14-2. Prescaler for synchronous Timer/Counters



AVR Timer/Counter Additional Configuration

- To use a prescaler of 1024, we must set `TCCR1B = 0x05;`
- The timer/counter also has to be set to operate as a timer instead of a counter!
- We do this by setting `TCCR1A = 0x00;`
- The full details (as always) are in the datasheet!
- Once correctly configured, the timer/counter register itself, `TCNT1`, will start counting up automatically at the specified rate.

Tick Calculations

Say we want to wait for 500 ms. The CPU (clock) frequency on the UCFK is 8 MHz. If the prescaler is 1024, how many clock ticks do we need to wait for?

Treat "ticks" as just another unit of time, like seconds or milliseconds.

$f_{\text{timer}} = \frac{f_{\text{clk}}}{\text{prescaler}} = \frac{8 \times 10^6}{1024} = 7812.5$ Hz.

Resolution = $\Delta t = \frac{1}{f_{\text{timer}}} = \frac{\text{prescaler}}{f_{\text{clk}}} = 128 \times 10^{-6}$ s $= 128$ µs.

This means our unit conversion is 128 $\frac{\mu\text{s}}{\text{tick}}$, or 7.8125 $\frac{\text{tick}}{\text{ms}}$.

Total ticks = $\frac{\text{Delay}}{\Delta t} = \frac{0.5}{128 \times 10^{-6}} = 3906$ ticks.

Reading & Writing the Timer/Counter Value

There are 2 timer/counters on the ATmega32U2 (`TCNT0` is 8 bit, `TCNT1` is 16 bit), which can be read from or written to at any time.

```
void timerDelay(uint16_t ticks)
{
    TCNT1 = 0;

    while (TCNT1 < ticks) {
        continue;
    }
}
```

- First, we reset the register `TCNT1` to 0.
- Then we wait until `TCNT1` reaches a specific value (in ticks).
- Once the function exits, approximately that many ticks have passed.

Integer Arithmetic

- AVR microcontrollers support only 8-bit signed and unsigned integers (not `floats` or `doubles` - don't use them!).
- Multi-byte integers can be implemented in software using carry bits. The compiler will handle this for you.
- We can use careful order of operations and unit changes to avoid needing `float`s, but beware:
  - **Overflow:** Occurs when any calculation (including intermediate steps) exceeds the maximum value for that precision.
  - **Underflow:** Occurs when any calculation (including intermediate steps) is too close to zero to be represented.

Integer Arithmetic Example

```c
#include <stdio.h>
#include <stdint.h>

int main(void)
{
    uint8_t x = 10; // All uint8_t, so values 0 through 255.
    uint8_t y = 50;
    uint8_t z = 255;

    uint8_t c = z + 1; // 255 + 1.
    printf("Overflow x = %u\n", c); // Outputs 0.

    uint8_t a = (x/y)*10;
    printf("Underflow a = %u\n", a); // Outputs 0.

    uint8_t b = (x*10)/y;
    printf("No underflow b = %u\n", b); // Outputs 2.
}
```

Delay Conversion: Millisecond to Ticks

- `ticks = delay *` ???
- Conversion rate: $\frac{\text{ticks}}{\text{s}} = \frac{1}{\Delta t} = \frac{f_{\text{clk}}}{\text{prescalar}}$.
- `delay` is in ms, so conversion rate = $\left(\frac{f_{\text{clk}}}{\text{prescalar}}\right) \div 1000$.
- In code:

```
ticks = delay * (F_CLK / PRESCALER) / 1000;
```

- Note that this ideally all happen with `uint16_t` numbers. Because `F_CLK` is 8000000 (larger than a `uint16_t`), 32-bit maths is used for the RHS.
- If we wanted everything to be 16-bit, we would instead use:

```
ticks = delay * (F_CLK / PRESCALER / 1000);
```

- Precision loss can be offset with different units if needed.

Example Delay Code

```c
#include <stdio.h>
#include <stdint.h>
#define F_CLK 8000000
#define TIMER1_PRESCALE 1024

int main(void)
{
    uint16_t delay = 500; // All 16-bit unsigned, values 0 through 65535.

    // Numerator and denominator overflow.
    uint16_t ticks = delay*F_CLK/(1000*TIMER1_PRESCALE);
    printf("Overflow ticks = %u\n", ticks); // 65248

    // Numerator underflows.
    uint16_t ticks2 = (delay/1000)*(F_CLK/TIMER1_PRESCALE);
    printf("Overflow ticks2 = %u\n", ticks2); // 0

    // Threads the needle; no overflow or underflow.
    uint16_t ticks3 = delay*(F_CLK/TIMER1_PRESCALE)/1000;
    printf("No overflow or underflow ticks3 = %u\n", ticks3); // 3906

}
```

Key Concepts

- Measuring time on a micro is needed even for the simplest programs.
- We use built-in peripherals called timer/counters to achieve this.
- The speed at which a timer counts up (how many ticks it makes in a second) is controlled by a prescaler.
- All this happens in hardware with integers, so we should use integer maths to handle timer values.
- This means we need to watch out for overflow and underflow!

Homework

Will the “working” tick calculation code work for all prescaler values that the AVR supports?

```
ticks = delay*(F_CLK/TIMER1_PRESCALE)/1000;
```

Why or why not?